Lab 4: VibeCoding — Brick Breaker

Complete AI Coding Chat History & Implementation Deliverables

Repository:	https://github.com/SETAPESU26/49_brick-breaker
Deliverable:	Lab-4 Chat History Export (Document/PDF)
Tasks Completed:	Task 1 (Collisions), Task 2 (Game Over), Task 3 (Replay/Difficulty), Task 4 (Sounds)

Turn 1 & 2: Task 1 — Refine Collision Detection

USER PROMPT:

Implement ONLY Task 1 from README.md: *Refine Collision Detection*. Current issue in game/game_engine.py: Paddle and brick collisions always flip vy unconditionally. Determine actual collision side using ball's previous position / movement direction or penetration depth. Reverse vx for horizontal-side hits, reverse vy for vertical-side hits. Prevent sticking inside paddle or bricks. Preserve score, lives, bricks, paddle controls.

ASSISTANT IMPLEMENTATION & RESOLUTION:

1. Analyzed game/game_engine.py and identified that both paddle and brick collisions called self.ball.vy *= -1 without checking collision direction.
2. Implemented _resolve_collision(self, rect) using previous frame coordinates (prev_x = ball.x - vx, prev_y = ball.y - vy) and Axis Penetration Depths (Minimum Translation Vector).
3. Reversed vx on horizontal hits (left/right) while preserving vy, and reversed vy on vertical hits (top/bottom) while preserving vx.
4. Repositioned the ball immediately onto the outer boundary of the obstacle to permanently prevent clipping or trapping.
5. Automated unit tests verified all 7 collision scenarios (Paddle top, left, right; Brick top, bottom, left, right).

```
def _resolve_collision(self, rect):
    ball_rect = self.ball.rect()
    if not ball_rect.colliderect(rect):
        return False
    prev_x, prev_y = self.ball.x - self.ball.vx, self.ball.y - self.ball.vy
    prev_rect = pygame.Rect(prev_x - self.ball.radius, prev_y - self.ball.radius,
                            self.ball.radius * 2, self.ball.radius * 2)
    from_above = prev_rect.bottom <= rect.top
    from_below = prev_rect.top >= rect.bottom
    from_left = prev_rect.right <= rect.left
    from_right = prev_rect.left >= rect.right
    ...
    if side in ("horizontal", "both"):
        hit_left = from_left or (not from_right and overlap_left < overlap_right)
        self.ball.vx = -abs(self.ball.vx) if hit_left else abs(self.ball.vx)
        self.ball.x = (rect.left - self.ball.radius) if hit_left else (rect.right + self.ball.radius)
    if side in ("vertical", "both"):
        hit_top = from_above or (not from_below and overlap_top < overlap_bottom)
        self.ball.vy = -abs(self.ball.vy) if hit_top else abs(self.ball.vy)
        self.ball.y = (rect.top - self.ball.radius) if hit_top else (rect.bottom + self.ball.radius)
    return True
```

Turn 3: Task 2 — Implement Game Over Condition

USER PROMPT:

Now implement ONLY Task 2 from README.md: *Implement Game Over Condition*. Detect WIN condition when all bricks destroyed. Detect LOSE condition when lives run out. Replace console-only messages with a proper Pygame graphical end screen showing 'YOU WIN!' or 'GAME OVER', along with 'Final Score: <score>'. Stop normal gameplay while displayed. Wait for keyboard input rather than immediately closing.

ASSISTANT IMPLEMENTATION & RESOLUTION:

1. Added self.title_font (Arial 48 bold) and self.sub_font (Arial 22) to GameEngine.
2. In render(), created a semi-transparent dark overlay (RGBA (10, 10, 20, 215)) that dims the playfield.
3. Centered title banner: green 'YOU WIN!' or red 'GAME OVER'.
4. Displayed 'Final Score: {self.score}' and 'Press any key to continue'.
5. In handle_event(), intercepted pygame.KEYDOWN events to gracefully signal continuation via pygame.QUIT.

Turn 4: Task 3 — Add Replay Option & Difficulty Selection

USER PROMPT:

Implement ONLY Task 3 from README.md: *Add Replay Option*. Show replay/difficulty selection on end screen with choices: Easy, Medium, Hard, Exit. Difficulty must actually change gameplay: Easy (slower ball, larger paddle), Medium (normal), Hard (faster ball, smaller paddle). Starting new game must reset score, lives, bricks, paddle, and ball. Selecting Exit closes gracefully.

ASSISTANT IMPLEMENTATION & RESOLUTION:

1. Defined concrete difficulty tuning parameters:
• **Easy**: Paddle width 130px (+30%), Ball speed 3 px/frame (-25%).
• **Medium**: Paddle width 100px (Default), Ball speed 4 px/frame (Default).
• **Hard**: Paddle width 70px (-30%), Ball speed 6 px/frame (+50%).
2. Added restart_game(difficulty) to rebuild bricks, reset score to 0, lives to 3, clear game_over flag, and spawn paddle/ball according to chosen difficulty.
3. Updated _reset_ball() to preserve the active difficulty speed across life losses.
4. Enhanced the end-screen UI with menu options [1] Easy, [2] Medium, [3] Hard, and [Q]/[ESC] Exit.
5. Handled keyboard selections in handle_event().

Turn 5 & 6: Task 4 — Add Sound Feedback & Audio Assets

USER PROMPT:

Implement Task 4: Add sound effects for brick destroyed, paddle hit, wall bounce, and win/game-over. Use Pygame mixer. Handle audio initialization safely so the game does not crash on systems without sound devices. Generate actual playable WAV files in assets/sounds/ (paddle.wav, brick.wav, wall.wav, win.wav, lose.wav) using Python standard library (wave, struct, math).

ASSISTANT IMPLEMENTATION & RESOLUTION:

1. Created generate_sounds.py using Python's standard wave and struct modules to synthesize 16-bit 44.1 kHz PCM audio files:
• paddle.wav: 80ms soft bounce blip (520Hz → 440Hz)
• brick.wav: 110ms higher-pitched crunchy break (780Hz → 320Hz)
• wall.wav: 50ms click/bounce (380Hz)
• win.wav: 440ms victory arpeggio (C5 → E5 → G5 → C6)
• lose.wav: 560ms descending game-over tone
2. Added _init_sounds() with try...except safety to ensure compatibility in headless environments without audio cards.

3. Integrated sound triggers into update() on wall bounce, paddle collision, brick collision, win, and game over.

Turn 7: Module Import & GameEngine Resolution

USER PROMPT:

Fix ImportError: cannot import name 'GameEngine' from 'game.game_engine'. Ensure GameEngine is top-level and not nested. Preserve all 4 tasks.

ASSISTANT IMPLEMENTATION & RESOLUTION:

1. Added dual relative/absolute import fallback in game/game_engine.py.
2. Declared explicit __all__ = ['GameEngine'].
3. Created game/__init__.py exporting GameEngine.
4. Cleaned stale .pyc cache and recompiled all modules. Verified from game.game_engine import GameEngine succeeds.

Automated Test Suite Results

Test Case	Target Area	Result
Paddle top collision	Task 1 Collisions	**PASS (vy reversed, unstuck)**
Paddle side collisions	Task 1 Collisions	**PASS (vx reversed, vy kept)**
Brick collisions	Task 1 Collisions	**PASS (destroyed, score +1)**
Win detection & overlay	Task 2 Game Over	**PASS (screen rendered)**
Lose detection & overlay	Task 2 Game Over	**PASS (screen rendered)**
Easy difficulty tuning	Task 3 Replay	**PASS (130px paddle, speed 3)**
Hard difficulty tuning	Task 3 Replay	**PASS (70px paddle, speed 6)**
All 5 audio assets loaded	Task 4 Sounds	**PASS (non-zero WAVs)**
Event audio triggers	Task 4 Sounds	**PASS (all 5 events triggered)**